

Unidad 1: Definición de Calidad de Software

Módulo 1: Atributos de Calidad de Software

1. ¿Qué es la calidad?

La calidad en el software no es un concepto absoluto, sino multidimensional. Los documentos la definen desde varios ángulos: idea clave calidad = combinación de atributos deseados + percepción de que cumple expectativas.

- **Definición de la IEEE:** Es el grado en que el software posee una combinación deseada de atributos y el cliente o usuario percibe que el producto cumple con sus expectativas. (Si el Manual dice A, el producto haga A. Si hace B no funciona).
- **Concepto de "Aptitud para el uso":** Desde la perspectiva del usuario, un software tiene calidad si le ayuda a resolver sus problemas de manera confiable, usable y sostenible en el tiempo. (Aptitud para el uso, me resuelve el problema).
- **Definición General y de Negocio:** Se entiende como la **adaptabilidad a las necesidades del cliente**. Hoy en día, la calidad es el motor de la propuesta de valor de una empresa. No es solo "hacerlo bien", sino cumplir con lo que el negocio requiere para ser competitivo.
- **La Visión de Pirsig:** Citado en los módulos, plantea que la calidad es "simple, inmediata y directa", pero difícil de definir porque es una experiencia de valor.
- **La Naturaleza del Software:** A diferencia de los objetos físicos (que tienen longitud, color o peso), el software es una **entidad intelectual**. Esto lo hace más difícil de caracterizar. La calidad aquí se refiere a características mensurables comparadas con estándares.
- **American Heritage Dictionary:** Define la calidad como una **característica o atributo de algo**. Se enfoca en rasgos medibles (como longitud, color o propiedades eléctricas) que pueden compararse con estándares conocidos.

El Costo del Error (Concepto Clave): La calidad no se agrega al final, se diseña desde el inicio. Detectar un error en la etapa de **Requisitos** cuesta solo una charla o corrección de documento (Costo 1x). Si el error llega a **Codificación**, el costo sube (10x). Pero si el error llega a **Producción**, el costo es máximo (100x o más) debido a la pérdida de confianza del cliente, soporte técnico y horas de retrabajo urgente.

Visión Multidimensional de la Calidad:

- **Para el Usuario:** Aptitud para el uso (¿me resuelve el problema?).
- **Para el Negocio:** Valor y competitividad (¿es rentable?).
- **Para el Desarrollador:** Mantenibilidad y código limpio.

2. Principios de Calidad de Software.

Los atributos son las características técnicas que podemos medir para determinar si un producto es de "calidad". Se dividen principalmente en:

▪ **Atributos Orientados al Producto (Externos e Internos)**

1. **Correctitud (Correctness):** El grado en que el software realiza las funciones para las que fue diseñado de acuerdo con la especificación ((Hace lo que debía hacer)).
2. **Confiabilidad (Reliability):** La capacidad del sistema de no fallar y de operar bajo condiciones específicas por un periodo determinado. Es la probabilidad de operación libre de fallos ((No falla cuando más lo necesitas)).
3. **Robustez:** La capacidad del software de seguir funcionando o degradarse con elegancia ante entradas inválidas o condiciones ambientales estresantes ((Tolera errores de entrada y casos raros)).
4. **Performance (Eficiencia):** El uso óptimo de recursos (CPU, memoria, disco) y la capacidad de respuesta en tiempos aceptables para el usuario ((Responde en tiempos razonables)).
5. **Usabilidad (Amigabilidad):** Qué tan fácil es para el usuario aprender a usar el sistema, operarlo y entender los resultados sin necesidad de formación extensa ((Se entiende sin manual de 200 páginas)).
6. **Mantenibilidad:** La facilidad con la que se pueden realizar cambios (correcciones, adaptaciones o mejoras) sin introducir nuevos errores. Es vital para la vida a largo plazo del software. ((Se puede corregir y mejorar sin romper todo)).
7. **Portabilidad:** La capacidad de ejecutar el software en diferentes plataformas (hardware, sistemas operativos o navegadores) sin cambios significativos ((Corre en distintos ambientes)).
8. **Interoperabilidad:** Habilidad de un sistema para convivir y cooperar con otros sistemas (ej. intercambio de datos vía APIs o estándares de interfaz), ((Sus partes sirven en otros contextos)).
9. **Integridad:** Atributo que garantiza el control de accesos no autorizados a los datos o al código, protegiendo la información.

▪ **Atributos Orientados al Proceso**

1. **Productividad:** Mide la eficiencia del equipo de desarrollo. Un proceso productivo entrega resultados más rápido usando menos recursos.
2. **Oportunidad (Timeliness):** La habilidad de entregar el producto en el tiempo pactado. Requiere una estimación precisa y metas claras.
3. **Visibilidad:** Un proceso es visible si todos sus pasos y su estado actual están claramente documentados y son accesibles para una auditoría externa en cualquier momento.

3. Calidad de Producto y Calidad de Proceso.

Existe una relación de dependencia crítica entre ambos conceptos:

- **Calidad de Proceso:** Se centra en "cómo" se construye. Incluye las herramientas, los métodos de gestión y las prácticas de ingeniería. Según Sommerville, un proceso de calidad es la base para un producto de calidad, ya que reduce la variabilidad y el error humano.
- **Calidad de Producto:** Se centra en el "qué" se entrega. Es la evaluación del resultado final frente a los atributos mencionados arriba.
- **La Premisa de la Cátedra: "La calidad no se agrega al final".** No es una capa de barniz que se pone cuando el código está terminado. Debe ser una actividad planificada que atraviesa todo el ciclo de vida del desarrollo.

Módulo 2: Aseguramiento de la Calidad del Software

1. Diferencia Aseguramiento de la calidad (QA) y el control de la calidad (QC).

Esta es una distinción fundamental para cualquier profesional del área:

Característica	Aseguramiento de la Calidad (QA)	Control de Calidad (QC)
Enfoque	Orientado al Proceso . ((Procesos del proyecto)).	Orientado al Producto . ((Contenido del producto)).
Naturaleza	Proactiva: busca prevenir que ocurran defectos.	Reactiva: busca encontrar defectos ya existentes.
Objetivo	Asegurar que se sigan los estándares, planes y procesos. ((Asegurar la adherencia a los procesos, estándares y planes)).	Evaluar si el producto es usable y cumple los requisitos. ((Detectar problemas en los productos de trabajo)).
Actividades	Auditorías, capacitación, definición de guías, monitoreo de procesos. ((Guía y monitoreo de los procesos utilizados y control de que las revisiones se llevan a cabo)).	Revisiones de código, inspecciones de documentos, ejecución de pruebas. ((Revisiones de productos de trabajo)).

Característica	Aseguramiento de la Calidad (QA)	Control de Calidad (QC)
Responsabilidad	Es una responsabilidad de gestión y de todo el equipo.	Es una evaluación independiente sobre la capacidad del producto.

2. Concepto de prueba de software y su relación con el aseguramiento de su calidad (QA).

- **¿Qué es el Testing?** Es el proceso de ejecutar un sistema con el fin de encontrar errores o verificar que cumple con lo esperado. Aporta **evidencia objetiva**.
- **Relación con QA:** El Testing es una actividad técnica que forma parte del Control de Calidad (QC). Los resultados del testing son el insumo que el QA utiliza para mejorar los procesos. Si el testing reporta muchos errores en una fase, QA debe analizar por qué el proceso permitió que esos errores se generaran (ej. mala definición de requerimientos).

3. Evaluación de calidad de Software

La evaluación se sustenta en dos pilares que a menudo se confunden:

- **Verificación:** "¿Estamos construyendo el producto correctamente?". Es un proceso interno que comprueba si el software cumple con las especificaciones técnicas y los estándares definidos en cada fase.
- **Validación:** "¿Estamos construyendo el producto correcto?". Es un proceso externo (hacia el cliente/usuario) que comprueba si el software realmente satisface las necesidades del usuario final.

Criterios para detectar problemas en la evaluación:

Al evaluar un producto de trabajo (documento o código), buscamos tres tipos de fallas:

- Omisión:** Falta algo que era necesario (un botón que no se puso).
- Excedente:** Hay cosas que no se pidieron y no agregan valor (funcionalidad extra innecesaria).
- Incorrecto:** Lo que está presente no funciona bien o es erróneo.

4. Establecimiento de métricas y normas de calidad

Para gestionar la calidad, es necesario medirla. Lo que no se mide, no se puede mejorar.

- **Métricas de Software:** Conjunto de medidas para estimar la calidad y planificar.
 - **Métricas de Control:** Evalúan el proceso (ej. "¿cuántos defectos encontramos por cada 1000 líneas de código?").
 - **Métricas de Predicción:** Intentan adelantar cómo será la calidad del producto (ej. medir la complejidad del diseño para predecir si será difícil de mantener).

- **Modelos y Estándares:**

- **ISO 9126 / ISO 25000:** Estándares internacionales que definen modelos de calidad de producto.
- **Modelo de McCall:** Organiza la calidad en factores (visión del usuario), criterios (visión del software) y métricas (medición técnica).
- **CMMI:** Un modelo de madurez de procesos que ayuda a las empresas a subir de nivel en su capacidad de desarrollo.

El Factor Costo (Concepto Crucial): Un punto clave mencionado en la "Clase 1" y en Pressman es el Costo de los Errores:

- Un error detectado en la fase de **Requerimientos** tiene un costo mínimo de corrección.
- Un error detectado en la fase de **Puesta en Régimen (Producción)** puede costar hasta 100 veces más, además del daño a la reputación y la pérdida de confianza del usuario.
- Por eso, el objetivo del QA es el "Shift Left": empezar a probar y asegurar la calidad lo más temprano posible en el proyecto.

Marcos Normativos e ISO

Esquema de Certificación y Madurez:

- **ISO/IEC 12207:** Norma para los procesos del **Ciclo de Vida** del software.
- **ISO/IEC 33000:** Método para evaluar la **calidad y madurez** de esos procesos.
- **CMMI (Capability Maturity Model Integration):** Modelo de madurez que permite a las empresas demostrar su nivel de capacidad mediante evidencia y auditorías.

Herramientas de Gestión de Calidad (Para análisis):

1. **Diagrama de Ishikawa (Causa-Efecto):** Para identificar la raíz de un problema.
2. **Ciclo PDCA (Deming):** Plan (Planificar), Do (Hacer), Check (Verificar), Act (Actuar). Es la base de la mejora continua.
3. **Los 5 Porqués:** Técnica para llegar al origen profundo de una falla preguntando sucesivamente "¿por qué?".
4. **Pareto (80/20):** El 80% de los defectos suelen concentrarse en el 20% de los módulos.

Módulo 3: Procedimientos y Certificaciones de Calidad

1. Proceso de Certificación

La certificación es una herramienta para demostrar la madurez de los procesos y la calidad de los productos de software. El esquema principal se basa en:

- **Normas base:** Se fundamenta en el modelo de procesos de la ISO/IEC 12207 y el método de evaluación de calidad y madurez ISO/IEC 33000.
- **Integración:** Está alineado con ISO 27001 (Seguridad) e ISO 25000 (Calidad de producto).

- **Resultados:** Al superar la auditoría, la organización obtiene el Certificado de Modelo de Madurez de la Ingeniería del Software y la licencia de uso de la marca correspondiente al nivel alcanzado.
- **Beneficios:** Diferenciación competitiva, reducción de fallos en producción, establecimiento de Acuerdos de Nivel de Servicio (SLA), ahorro de costos en mantenimiento y garantía de seguridad (confidencialidad, integridad, etc.).

2. Auditorías

Basadas en la norma ISO 19011, son procesos sistemáticos e independientes para evaluar evidencias objetivas.

- **Criterios:** Conjunto de políticas o requisitos usados como referencia.
- **Evidencias:** Registros o declaraciones de hechos verificables.
- **Hallazgos:** Resultados de evaluar la evidencia contra los criterios.
- **Plan vs. Programa:** El Programa es el conjunto de auditorías planificadas para un tiempo determinado, mientras que el Plan describe las actividades específicas en el sitio para una auditoría concreta.

3. Herramientas para la Gestión de Calidad del Software

Se utilizan diversas técnicas para identificar causas raíz y planificar mejoras:

- **Diagrama de Ishikawa (Causa y Efecto):** Identifica las causas de un problema específico.
- **Ciclo PDCA (Plan-Do-Check-Act):** Metodología para la mejora continua y resolución sistemática de problemas.
- **5 Porqués:** Técnica para llegar a la causa principal preguntando "por qué" sucesivamente.
- **Diagrama de Pareto:** Utilizado para priorizar los problemas (regla 80/20).
- **5W2H: Plan de acción que define:** Qué (What), Quién (Who), Cuándo (When), Dónde (Where), Por qué (Why), Cómo (How) y Cuánto (How much).
- **Brainstorming:** Fase de generación de ideas creativas para soluciones.

4. Técnicas y Metodología sobre Madurez del Proceso

El foco está en evaluar qué tan capaz es una organización de repetir sus éxitos:

- **CMMI (Capability Maturity Model Integration):** Integra disciplinas de software y sistemas en un marco de mejora para proveer estabilidad.
- **Evaluación de Procesos:** Su objetivo es conocer la capacidad actual de los procesos de una organización y determinar hasta qué punto cumplen su propósito.
- **ISO/IEC 15504 (SPICE):** Antecesora de la serie 33000, establece requisitos mínimos para evaluaciones consistentes.

5. Costos de la Calidad y Certificación ISO 25000

La ISO 25000 (SQuaRE) se divide en varias ramas (Gestión de Calidad, Modelo de Calidad, Medición, Requisitos y Evaluación) para asegurar la calidad del producto y los datos.

- **Costos de la Calidad:** El mayor impacto económico positivo se da en el ahorro de costos en mantenimiento al detectar y eliminar defectos antes de la entrega.
- **Recursos:** Es crítico balancear los objetivos de calidad con los recursos disponibles para evitar fallas en la implementación del modelo.

Unidad 2: Administración de Configuración (CM)

Módulo 4: Administración de configuración

Conceptos Fundamentales de GCS (Diferencias):

- **Ítem de Configuración:** Cualquier pieza de software o documento (ej. un .docx, un código fuente).
- **Versión:** Una instancia de un ítem en un momento dado.
- **Revisión:** Un pequeño cambio dentro de una versión.
- **Baseline (Línea Base):** Un punto de referencia ya aprobado y "congelado" para construir sobre él.
- **Release (Entrega):** El conjunto de ítems que se entregan al usuario final.

Plan GCS según IEEE 828 (Las 4 preguntas de Alessio):

1. **¿Quién?:** Roles y responsabilidades (Matriz de responsabilidades).
2. **¿Qué?:** Identificación de ítems, control de cambios y auditorías.
3. **¿Cuándo?:** Hitos, fechas y cronograma de releases.
4. **¿Cómo?:** Herramientas (Git, Jira, etc.) y ambientes de trabajo.

1. ¿Qué es Administración de Configuración?

Es la disciplina que provee estabilidad a la producción de software controlando la evolución del producto y sus cambios.

- **Definición IEEE:** Disciplina dedicada a identificar y documentar las características de un ítem de configuración, controlar cambios, registrar estados y verificar el cumplimiento de requerimientos.
- **Importancia:** Actúa como el centro de todas las actividades del proyecto (la "rueda del carro"), permitiendo la trazabilidad y evitando que los desarrolladores se "pisen" el trabajo.

2. Proceso de Administración de Configuración

Se compone de cinco partes fundamentales según el material:

1. **Planificación:** Define qué ítems se gestionarán, roles, responsabilidades, políticas y herramientas (como el plan IEEE 828) .
2. **Identificación:** Selección de ítems y definición de una convención de nombres única.

3. **Control de Cambios:** Gestión de solicitudes de cambio (PC) y aprobación a través del CCB (Change Control Board).
4. **Estado de la Configuración:** Registro de qué cambió, quién, cuándo y por qué.
5. **Auditoría de Configuración:** Verificación de que el producto se construyó según lo planeado.

3. Control de Versión y Línea Base (Baseline)

- **Ítem de Configuración (IC):** Cualquier elemento (plan, diseño, código, datos de prueba) que necesite ser controlado.
- **Versiones:** Se gestionan mediante mecanismos de Check-out (extraer para editar), Check-in/Commit (reingresar) y Branching/Merging (ramas paralelas y fusión).
- **Línea Base (Baseline):** Una versión aprobada de los ítems de configuración que sirve como punto de partida para el desarrollo posterior.
- **Esquema de Promoción:** El software pasa por distintos ambientes: Dev → Testing → Aceptación (UAT) → Producción.

4. Auditoría y Reporte de Estado

- **Reporte de Estado:** Provee visibilidad sobre la evolución del sistema. Incluye qué componentes han sido modificados y la historia de los cambios realizados.
- **Auditorías de CM:**
 - Auditoría Funcional: Comprueba si el ítem cumple con sus requerimientos.
 - Auditoría Física: Verifica si el ítem fue construido según la documentación técnica y si el paquete de release está completo.
- **Trazabilidad:** Permite reconstruir cómo llegó una versión específica a producción, garantizando que no se desplieguen versiones viejas o incorrectas por error.

Unidad 3: Fundamentos del Testing

Módulo 5: Fundamentos del testing

1. Definición de Testing (IEEE)

- Es el proceso de operar un sistema o componente bajo condiciones determinadas, observar y registrar los resultados, y evaluar los mismos.
- **Objetivo:** Identificar fallas, reducir riesgos y aumentar la confianza en el software.

2. Validación vs. Verificación (V&V)

- **Verificación:** "¿Estamos construyendo el producto correctamente?". Evalúa si el software cumple con las especificaciones técnicas (fase por fase).
- **Validación:** "¿Estamos construyendo el producto correcto?". Evalúa si el software satisface las necesidades reales del cliente/usuario final.

3. La Cadena del Problema

- **Error:** Equivocación humana (error del programador o analista).
- **Defecto (Bug):** Desperfecto físico en el código o documento producto de un error.
- **Falla:** Manifestación externa o comportamiento incorrecto del sistema al ejecutarse el código con defectos.

4. Axiomas del Testing

- La prueba exhaustiva es imposible.
- Un buen test es aquel que tiene alta probabilidad de encontrar un defecto.
- El programador no debe testear su propio código (visión túnel).
- Es obligatorio definir un **resultado esperado** antes de la ejecución.
- El testing debe empezar lo antes posible en el ciclo de vida.

5. Niveles de Testing (Modelo-V)

- **Pruebas Unitarias:** Verifican módulos o funciones individuales.
- **Pruebas de Integración:** Verifican la interacción entre módulos.
- **Pruebas de Sistema:** Verifican el sistema completo frente a los requisitos.
- **Pruebas de Aceptación:** Validación final realizada por el usuario.

Unidad 4: Técnicas de diseño de casos de test

Módulo 6: Diseño de casos de test

1. Definición de Caso de Prueba

- Conjunto de: **Datos de entrada + Condiciones de ejecución + Resultado esperado.**
- **Criterio:** Encontrar la mayor cantidad de defectos con el menor número de ensayos (optimización de recursos).

La Trilogía del Fallo:

1. **Error:** Una equivocación humana (del analista al escribir el requisito o del dev al programar).
2. **Defecto:** El problema físico oculto en el producto (en el código o documento).
3. **Falla:** El comportamiento incorrecto que ocurre cuando ejecutamos el software y el defecto se manifiesta.

Técnicas Inteligentes de Diseño (Caja Negra):

- **Partición de Equivalencia:** Agrupar datos en clases (válidas e inválidas) y probar un solo representante de cada clase para no hacer pruebas infinitas.
- **Análisis de Valores Límite (Borde):** Probar los extremos de los rangos (ej: si el sistema acepta de 1 a 100, probar 0, 1, 2 y 99, 100, 101). Aquí es donde más errores aparecen.
- **Error Guessing:** Técnica basada en la experiencia y la intuición del tester para predecir dónde fallará el sistema (campos vacíos, nulos, negativos).

2. Técnicas Estáticas (Sin ejecución)

- **Análisis de código:** Revisiones manuales (inspecciones o walkthroughs) en grupo.
- **Análisis automatizado:** Herramientas que buscan errores sintácticos o lógicos sin correr el programa.
- **Análisis formal:** Modelos matemáticos para probar correctitud en sistemas críticos.

3. Técnicas Dinámicas (Con ejecución)

- **Caja Blanca (Estructural):** Se basa en el conocimiento del código interno (camino, bucles, condiciones).
- **Caja Negra (Funcional):** Se basa en los requisitos. Se ignora el código y se evalúan solo entradas y salidas.

4. Técnicas de Caja Negra (Selección de datos)

- **Clases de Equivalencia:** Dividir el dominio de entrada en grupos que el sistema trata igual. Se elige un valor representante por cada clase (válida e inválida).
- **Análisis de Valores de Borde:** Probar los límites de las clases de equivalencia (el valor exacto, uno antes y uno después), ya que ahí suelen esconderse los errores.
- **Error Guessing:** Uso de la intuición y experiencia para predecir dónde fallará el sistema (ej. valores nulos, división por cero).

5. Complejidad Ciclomática (Métrica de Caja Blanca)

- Mide la complejidad lógica y el número de caminos independientes.
- **Fórmulas:**
 1. $\$V(G) = \text{Aristas} - \text{Nodos} + 2\$$
 2. $\$V(G) = \text{Nodos} \setminus \text{Predicado} + 1\$$ (Los nodos predicados son los puntos de decisión como IF o WHILE).

Tip extra para el examen:

La Paradoja del Pesticida: Si repetís siempre los mismos casos de prueba, llegará un momento en que no encontrarán nuevos defectos. Por eso, los casos de prueba deben revisarse y actualizarse periódicamente.

La Falacia de "Ausencia de Errores": Que un sistema no tenga errores no significa que sea útil. Si el sistema es perfecto pero no hace lo que el usuario necesita, no tiene calidad. (Recordar: **Validación**).

El Principio de Independencia: El programador **no** debe testear su propio código. El tester necesita "ojos frescos" y no estar sesgado por cómo se construyó la solución.

Testing de Regresión (El efecto dominó): Cada vez que se arregla un bicho o se agrega algo nuevo, hay que volver a probar lo que ya funcionaba. ¿Por qué? Porque un cambio en un lado puede romper algo que no tenía nada que ver.

¿Cuándo se deja de testear? (Pregunta trampa): Nunca se termina de encontrar todo. Se deja de testear cuando: Se agota el tiempo o el presupuesto. Se cumplen los "Criterios de Salida" del plan. El riesgo de fallos es lo suficientemente bajo para el negocio.

Agrupamiento de Defectos: Los errores no están repartidos parejos. Suelen amontonarse. Si encontrás un bicho en un módulo, es muy probable que haya diez más escondidos ahí cerca. (Relacionarlo con el **Principio de Pareto 80/20**).

Diferencia entre Dato de Prueba y Caso de Prueba:

- **Caso de prueba:** Es el escenario completo (Pasos + Resultado Esperado).
- **Dato de prueba:** Es el valor específico que ingresás (ej: "Juan123")

¿Qué es lo más importante de un reporte de error? la respuesta es "La Reproducibilidad". Si el programador no puede repetir el error siguiendo tus pasos, no lo puede arreglar.

El "Oráculo" de Pruebas: Para que un test sea válido, **siempre** debe existir un "resultado esperado" antes de ejecutarlo. Si no sabés qué debería pasar, no estás testeando, estás explorando. El test vive de la comparación: *Esperado vs. Obtenido*.

Auditoría Funcional vs. Física (Clase GCS):

- **Funcional:** Verifica si el ítem de configuración hace lo que dice su documentación técnica (¿funciona como dice el manual?).
- **Física:** Verifica si el ítem está completo y si todos los componentes necesarios están presentes en el release (¿están todos los archivos que deben estar?).

La Promoción de Ambientes (El camino a Producción): Nunca se pasa de "mi máquina" a Producción. La secuencia segura es: **Dev** (Desarrollo) -> **Testing** (QA) -> **Aceptación** (Usuario/UAT) -> **Producción**. Cada salto debe dejar una **evidencia** (registro).

¿Por qué el software es una "Entidad Intelectual"? A diferencia de un auto o una casa, el software no se gasta con el uso. Su "deterioro" es en realidad **obsolescencia** o errores de lógica no descubiertos. Por eso la calidad se mide en atributos abstractos (usabilidad, mantenibilidad, portabilidad).

Criterios de Entrada y Salida: Entrada: ¿Qué necesito para empezar a testear? (ej: Código estable, ambiente listo, casos de prueba escritos). Salida: ¿Cuándo puedo decir "terminé"? (ej: Se ejecutaron todos los casos críticos, no quedan errores de prioridad alta, se acabó el tiempo).

La Matriz de Responsabilidades: En un Plan de Gestión de Configuración (IEEE 828), no alcanza con decir qué hay que hacer; hay que decir **quién** es el responsable de aprobar cada cambio. Sin responsable, no hay control.

TESTING ESTÁTICO VS. DINÁMICO:

- **Testing Estático (Revisiones):** Se hace **sin ejecutar** el programa. Consiste en revisar los documentos de requerimientos, el diseño, la arquitectura o el código fuente (inspecciones, "walkthroughs" o caminatas).
 - **Pro Tip:** Es el testing más barato porque encontrás el error *antes* de que nazca. Si corregís un error en el papel, evitás que se programe mal.
- **Testing Dinámico (Ejecución):** Es el que todos conocen; requiere **ejecutar** el software (darle "play") para ver cómo reacciona ante los datos. Aquí entran las pruebas de Caja Negra y Caja Blanca.

Unidad 5: Estrategias de testing

Módulo 7: Estrategias de testing

5. Gestión de riesgo en testing.
6. Prueba unitaria
7. Prueba de integración.
8. Prueba de sistema.
9. Prueba de desempeño (Performance, Carga, Stress, usabilidad, etc.).
10. Prueba de aceptación.
11. Prueba de Regresión.

Unidad 6: Plan de Testing

Módulo 8: Plan de Testing

12. Herramientas de gestión.
13. Analizar el contenido de un documento de plan de pruebas.
14. Estimación del testing
15. Test Case Point Analysis
16. Objetivos y alcances de los ambientes de test.
17. Creación de un ambiente de test.
18. Control de un ambiente de test.
19. Restauración de un ambiente de test.
20. Plan de mediciones.
21. Tablero de control de medición de testing.

Unidad 7: Principios del testing Ágil

Módulo 9: Principios del Testing ágil

22. El cambio estratégico: Técnicas tradicionales vs ágil.
23. ¿Qué es test agile?
24. Proceso de testing agile.
25. Cuadrante de agile testing.
26. Test-Driven Development (TDD).
27. Acceptance Test-Driven Development (ATDD).

28. Behavior-Driven Development (BDD).

Unidad 8: DevOps y Testing Web y Mobile

Módulo 10: Devops

- 29. ¿Qué es DevOps y DevSegOps?
- 30. Testing automatizado
- 31. Tipos de Herramientas de Testing.
- 32. Integración y Despliegue continuo
- 33. Revisión de código (Calidad),
- 34. Puesta en producción
- 35. Mantenimiento como un proceso automatizado.
- 36. Evaluación de Herramientas de Testing

Módulo 11: Testing Web y Mobile

- 37. Conceptos de testing en aplicaciones WEB.
- 38. Test de interfaz de usuario.
- 39. Test de navegación. Herramientas de Testing Web.
- 40. Testing Mobile.
- 41. Testing Web vs Testing Tradicional vs Mobile

Unidad 9: Estándares Testing

Módulo 12: Estándares Testing

- 42. IEEE Std829, Software Test Documentation, IEEE Std1008, Software UnitTesting.
- 43. Elaboración y estructura de ISO/IEC/IEEE 29119 Software
- 44. Testing: Conceptos y definiciones
- 45. Modelo de procesos de pruebas
- 46. Documentación de pruebas

Módulo 13: TMMI

- 47. Técnicas de prueba. Test Maturity Model Integration (TMMi).
- 48. TOM Test organization Maturity.
- 49. TIM Test improvement Model.
- 50. TPI next Test Process Improvement.

51. TMAP Next Test Management Approach.